

Hw3

zhijing huang

2026-09-13

Question 2

- The high correlation between columns shows that the sum is a well-observed direction with identical predictors. Because $X_1 - X_2$ is close to 0, their difference is a weak direction with a small singular value.
- Ridge can substantially stabilize the individual coefficients without producing an equally large change in the fitted values because Ridge reduces estimation variance, and stabilizes the division between β_{hats} .

```
set.seed(43231)
n = 100
p = 4
repetitions = 200

L <- rnorm(n)
U1 <- rnorm(n)
U2 <- rnorm(n)
X_raw3 <- rnorm(n)
X_raw4 <- rnorm(n)

X_raw <- cbind(
  L + 0.04 * U1,
  L + 0.04 * U2,
  X_raw3,
  X_raw4
)

x_bar <- colMeans(X_raw)
x_centered <- sweep(X_raw, 2, x_bar, "-")
s <- sqrt(colMeans(x_centered^2))
X <- sweep(x_centered, 2, s, "/")
colMeans(X)
```

```
##
##           X_raw3      X_raw4
## 1.297573e-17 -2.331468e-17 -3.164136e-17 -2.220446e-18
```

```
colMeans(X^2)
```

```
##
##           X_raw3 X_raw4
##           1      1      1      1
```

```
dim(X)
```

```
## [1] 100  4
```

```
beta <- c(1.5, 1.5, 1, 0)
mu <- drop(X %*% beta)
lambda_grid <- c(0, 0.02, 0.2)

beta_hat <- array(NA_real_, c(repetitions, p, length(lambda_grid)))
train_mse <- matrix(NA_real_, repetitions, length(lambda_grid))
test_mse <- matrix(NA_real_, repetitions, length(lambda_grid))

for (k in seq_len(repetitions)) {
  y <- mu + rnorm(n)
  y_test <- mu + rnorm(n)
  y_centered <- y - mean(y)

  for (l in seq_along(lambda_grid)) {
    lambda <- lambda_grid[l]
    beta_hat[k, l] <- solve(
      crossprod(X) + n * lambda * diag(p),
      crossprod(X, y_centered)
    )
    y_hat <- mean(y) + drop(X %*% beta_hat[k, l])

    train_mse[k, l] <- mean((y - y_hat)^2)
    test_mse[k, l] <- mean((y_test - y_hat)^2)
  }
}
correlation_i2 <- cor(X[, 1], X[, 2])
correlation_i2
```

```
## [1] 0.9977057
```

```
singular_values <- svd(X)$d
length(singular_values)
```

```
## [1] 4
```

```
singular_values[1]
```

```
## [1] 14.14148
```

```
singular_values[length(singular_values)]
```

```
## [1] 0.4753575
```

```
beta1_values <- beta_hat[, 1, 1]
beta2_values <- beta_hat[, 2, 1]
beta_sum_values <- beta1_values + beta2_values
beta_difference_values <- beta1_values - beta2_values
```

```
coefficient_quantities <- cbind(
  beta1 = beta1_values,
  beta2 = beta2_values,
  beta_sum = beta_sum_values,
  beta_difference = beta_difference_values
)
```

```
colMeans(coefficient_quantities)
```

```
##          beta1          beta2          beta_sum beta_difference
##    1.49558941    1.50976489    3.00535430    -0.01417548
```

```
apply(coefficient_quantities, 2, sd)
```

```
##          beta1          beta2          beta_sum beta_difference
##    1.54394848    1.54858847    0.09510778     3.09107762
```

```
coefficient_summary <- data.frame(
  quantity = colnames(coefficient_quantities),
  mean = colMeans(coefficient_quantities),
  standard_deviation = apply(coefficient_quantities, 2, sd),
  row.names = NULL
)
seq_along(lambda_grid)
```

```
## [1] 1 2 3
```

```
average_train_mse <- colMeans(train_mse)
average_test_mse <- colMeans(test_mse)

print(c(
  correlation = cor(X[, 1], X[, 2]),
  largest_singular_value = max(singular_values),
  smallest_singular_value = min(singular_values)
), digits = 4)
```

```
##          correlation largest_singular_value smallest_singular_value
##          0.9977          14.1415          0.4754
```

```
knitr::kable(
  coefficient_summary,
  caption = "Empirical mean (standard deviation) across 200 repetitions"
)
```

Table 1: Empirical mean (standard deviation) across 200 repetitions

quantity	mean	standard_deviation
beta1	1.4955894	1.5439485
beta2	1.5097649	1.5485885
beta_sum	3.0053543	0.0951078
beta_difference	-0.0141755	3.0910776

```
average_train_mse
```

```
## [1] 0.9647563 0.9746809 1.0767603
```

```
average_test_mse
```

```
## [1] 1.062452 1.049262 1.133484
```

Question 3

- a. Because after differentiating

$$\nabla L_\lambda(\beta)$$

by beta, and set the gradient to 0, and then solve for beta_hat, we'll get:

$$\hat{\beta}_\lambda = (\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p)^{-1} \mathbf{X}^\top \tilde{\mathbf{y}}.$$

For every nonzero vector $\mathbf{a} \in \mathbb{R}^p$,

$$\mathbf{a}^\top (\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p) \mathbf{a} = \|\mathbf{X}\mathbf{a}\|_2^2 + n\lambda \|\mathbf{a}\|_2^2 > 0$$

- b. As the matrix is invertible when gamma is greater than 0,

$$\hat{\beta}_\lambda = \mathbf{V}_r (\mathbf{D}_r^2 + n\lambda \mathbf{I}_r)^{-1} \mathbf{D}_r \mathbf{U}_r^\top \tilde{\mathbf{y}}.$$

$$\mathbf{X} \hat{\beta}_\lambda = \sum_{j=1}^r \frac{d_j^2}{d_j^2 + n\lambda} \mathbf{u}_j \mathbf{u}_j^\top \tilde{\mathbf{y}}.$$

plug in $n\lambda = 1$,

$$\rho_1 = \frac{100}{101} \approx 0.9901,$$

$$\rho_2 = \frac{4}{5} = 0.8,$$

$$\rho_3 = \frac{0.04}{1.04} \approx 0.0385.$$

The coefficient is $1+0.991+0.8+0.0385 = 2.8286$.

- c. the third direction is the strongest, as it has the smallest value. However, shrinkage reduces the ability to response to noise, so if the true mean response has a large component in the third left singular-vector direction, reducing it by 0.0385 will the same shrinkage produce substantial prediction bias.

Question 4

- a.

$$\kappa(\mathbf{A}_0) = \frac{9}{0.01} = 900.$$

Adding 0.09 to both values, we get 9.09 and 0.1.

$$\kappa(\mathbf{A}_\lambda) = \frac{9.09}{0.10} = 90.9.$$

The ridge objective will be more curved.

- b.

$$\mathbf{e}^{(k)} = \beta^{(k)} - \hat{\beta}_\lambda.$$

$$\nabla L_\lambda(\beta^{(k)}) = \mathbf{A}_\lambda \mathbf{e}^{(k)}.$$

$$\mathbf{e}^{(k+1)} = (\mathbf{I}_p - \eta \mathbf{A}_\lambda) \mathbf{e}^{(k)}.$$

$\mathbf{A}_\lambda$ has eigenvalue $a + \lambda$. The corresponding error component is therefore multiplied by $1 - \eta(a + \lambda)$.

Every direction contracts when

$$|1 - \eta(a + \lambda)| < 1$$

$2/M = 2/9.09 = 0.22$, For $\eta = 1/M$, the contraction factors are $1-9.09/9.09 = 0$ in the strong direction, and $1-0.1/9.09 = 0.989$ in the weak direction. by $(1-0.1/9.09)^3 = 0.9674$, and for $\eta = 2.1/M$,

$$1 - \frac{2.1(9.09)}{9.09} = -1.1.$$

- c. The reason for λ to make gradient descent converge as quickly as possible is not a valid statistical tuning rule is because lambda introduces more shrinkage bias, and resulted in a large penalty, which is what we are trying to avoid.

Question 5

- a. lambda_min is 0.0199, lambda_1se is 1.5849, lambda_gcv is 0.0282.
b. Among the theww, OLS has the smallest training MSE, followed by CV_min, GCV, and One SE has the largest MSE. Because GCV estimates prediction error from the training fit by correcting training residuals by effective degrees of freedom, it is dependent. Ridge doesn't correct model misspecification, which makes OLS a best choice.

```

realestate <- read.csv("data/realestate.csv")
split_folds <- read.csv("data/realestate-split-folds.csv")
match_index <- match(split_folds$row_id, realestate$row_id)

coursedata <- cbind(split_folds, realestate[
  match_index,
  setdiff(names(realestate), "row_id"),
  drop = FALSE
])
feature_names <- c(
  "date", "age", "distance",
  "stores", "latitude", "longitude"
)
X_raw_all <- as.matrix(coursedata[, feature_names])
y_all <- coursedata$price
is_train <- coursedata$split == "train"
X_train_raw <- X_raw_all[is_train, , drop = FALSE]
y_train <- y_all[is_train]
X_test_raw <- X_raw_all[!is_train, , drop = FALSE]
y_test <- y_all[!is_train]
folds <- coursedata$cv_fold[is_train]

ridge <- function(X_raw, y, lambda, X_raw_new = NULL){
  n = nrow(X_raw)
  p = ncol(X_raw)
  x_bar = colMeans(X_raw)
  X_centered = sweep(X_raw, 2, x_bar, "-")
  s = sqrt(colMeans(X_centered^2))
  X = sweep(X_centered, 2, s, "/")
  y_bar = mean(y)
  y_centered <- y - y_bar
  beta_hat = solve(crossprod(X) + n * lambda * diag(p), crossprod(X, y_centered))

  beta_hat_raw <- drop(beta_hat) / s
  raw_intercept <- y_bar - sum(x_bar * beta_hat_raw)
  y_hat <- raw_intercept + drop(X_raw %*% beta_hat_raw)

  singular_values <- svd(
    X,
    nu = 0,
    nv = 0
  )$d
  df_effective <- 1 + sum(
    singular_values^2 /
    (singular_values^2 + n * lambda)
  )

  y_hat_new <- NULL
  if (!is.null(X_raw_new)) {
    y_hat_new <- raw_intercept + drop(X_raw_new %*% beta_hat_raw)
  }

  list(
    y_hat = y_hat,
    y_hat_new = y_hat_new,
    df_effective = df_effective
  )
}

lambda_grid <- c(0, 10^seq(-4, 2, length.out = 121))

fold_levels <- sort(unique(folds))
fold_mse <- matrix(
  NA_real_,
  nrow = length(fold_levels),
  ncol = length(lambda_grid)
)

for (k in seq_along(fold_levels)) {
  validation <- folds == fold_levels[k]

  X_train <- X_train_raw[!validation, , drop = FALSE]
  X_val <- X_train_raw[validation, , drop = FALSE]
  y_train_fold <- y_train[!validation]
  y_val_fold <- y_train[validation]

  x_bar <- colMeans(X_train)
  s <- sqrt(colMeans(sweep(X_train, 2, x_bar, "-")^2))

  X <- sweep(X_train, 2, x_bar, "-")
  X <- sweep(X, 2, s, "/")

  X_val <- sweep(X_val, 2, x_bar, "-")
  X_val <- sweep(X_val, 2, s, "/")

  y_bar <- mean(y_train_fold)
  y_centered <- y_train_fold - y_bar

  n <- nrow(X)
  p <- ncol(X)

  gram <- crossprod(X)
  rhs <- crossprod(X, y_centered)

  fold_mse[k, ] <- vapply(lambda_grid, function(lambda) {
    beta_hat <- solve(
      gram + n * lambda * diag(p),
      rhs
    )
    y_hat <- y_bar + drop(X_val %*% beta_hat)
    mean((y_val_fold - y_hat)^2)
  }, numeric(1))
}

cv_mean <- colMeans(fold_mse)
cv_se <- apply(fold_mse, 2, sd) / sqrt(nrow(fold_mse))
min_index <- which.min(cv_mean)
lambda_min <- lambda_grid[min_index]
one_se_limit <- cv_mean[min_index] + cv_se[min_index]
lambda_1se <- max(lambda_grid[cv_mean <= one_se_limit])

```

```

x_bar <- colMeans(X_train_raw)
X_centered <- sweep(X_train_raw, 2, x_bar, "-")
s <- sqrt(colMeans(X_centered^2))
X <- sweep(X_centered, 2, s, "/")
y_bar <- mean(y_train)
y_centered <- y_train - y_bar

```

```

n <- nrow(X)
p <- ncol(X)
gram <- crossprod(X)
rhs <- crossprod(X, y_centered)
singular_values <- svd(X, nu = 0, nv = 0)$d

gcv <- numeric(length(lambda_grid))
df_eff_path <- numeric(length(lambda_grid))
for (l in seq_along(lambda_grid)) {
  lambda <- lambda_grid[l]
  beta_hat <- solve(gram + n * lambda * diag(p), rhs)
  y_hat <- y_bar + drop(X %*% beta_hat)
  df_eff_path[l] <- 1 + sum(singular_values^2 / (singular_values^2 + n * lambda))
  training_mse <- mean((y_train - y_hat)^2)
  gcv[l] <- training_mse /
    (1 - df_eff_path[l] / n)^2
}
lambda_gcv <- lambda_grid[which.min(gcv)]

positive_lambda <- lambda_grid > 0
plot_position <- numeric(length(lambda_grid))
plot_position[positive_lambda] <- log10(lambda_grid[positive_lambda])
plot_position[!positive_lambda] <- min(plot_position[positive_lambda]) - 1

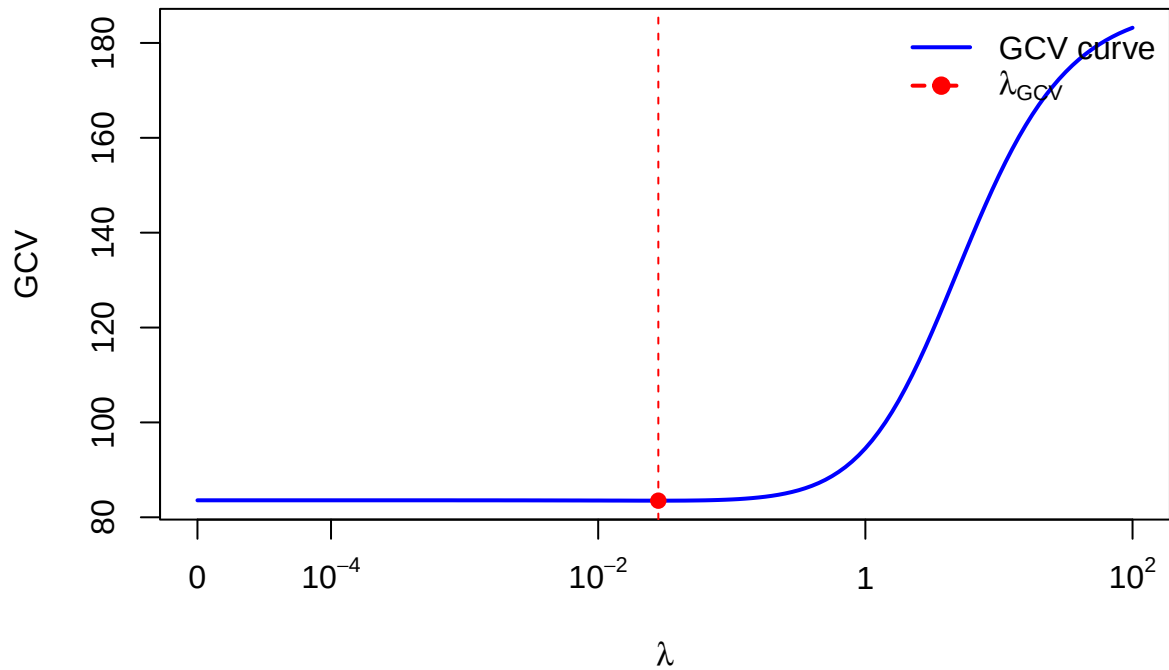
plot(
  plot_position,
  gcv,
  type = "l",
  lwd = 2,
  col = "blue",
  xaxt = "n",
  xlab = expression(lambda),
  ylab = "GCV"
)

axis(
  1,
  at = c(-5, -4, -2, 0, 2),
  labels = c("0", expression(10^-4), expression(10^-2), "1", expression(10^2))
)

gcv_index <- which.min(gcv)
points(plot_position[gcv_index], gcv[gcv_index], pch = 19, col = "red")
abline(v = plot_position[gcv_index], lty = 2, col = "red")

legend(
  "topright",
  legend = c("GCV curve", expression(lambda[GCV])),
  col = c("blue", "red"),
  lty = c(1, 2),
  pch = c(NA, 19),
  lwd = 2,
  bty = "n"
)

```



explain-to-me skill

--- name: explain-to-me description: Use when a student explicitly asks for help understanding a STAT 432 homework question without requesting its solution. ---

Intent

Help me understand a homework question while leaving the mathematical and computational work for me to complete.

Instructions

- Read the complete question before responding.
- Explain the purpose of the question in plain language.
- Preserve the notation used in the question.
- For Questions 2 and 3, use Ridge Regression: Stability Through Shrinkage.
- For Questions 4 and 5, use From a Penalized Objective to a Fitted Ridge Model. Point me to one relevant section within that page and explain briefly why it will help. Do not invent a section if you cannot access the page.
- Suggest a reasonable first step, then ask what part remains unclear.
- Do not carry out the calculations or derivations, write solution code, or give me the final answer.